

ATIVIDADE - DEPURAÇÃO DO JSOUP COM A JAGUAR PORTAL

EXPERIMENTO *DEPURAÇÃO DE PROGRAMAS E A PRÁTICA DE
INTEGRAÇÃO CONTÍNUA*

Erickson Lima, Marcos Lordello Chaim

Escola de Artes, Ciências e Humanidades | EACH | USP

Localização de Defeitos Baseado em Espectros, em inglês, *Spectrum-based Fault Localization (SBFL)*, utiliza a cobertura de código dos testes executados, isto é, os espectros, para ranquear partes do código que possuem maior probabilidade de estarem associados a uma falha. O *ranking* gerado pode orientar o desenvolvedor na sua busca por defeitos.

Para um melhor entendimento sobre a técnica de *SBFL*, explicaremos os seguintes conceitos:

- Defeito, Infecção e Falha;
- Fluxo de Controle;
- Espectro (Cobertura);
- Métricas de Associação.

DEFEITO, INFECÇÃO E FALHA

Defeito

Código que pode se comportar de maneira inesperada.

Infecção

Causado por um código com defeito.

Falha

Falha observada por um usuário em um código infectado por um defeito.

```
1 public static boolean toBoolean(String str) {
2     // Previously used equalsIgnoreCase, which was fast for interned 't
3     // Non interned 'true' matched 15 times slower.
4     //
5     // Optimisation provides same performance as before for interned 't
6     // Similar performance for null, 'false', and other strings not len
7     // 'true'/'TRUE' match 4 times slower, 'TRUE'/'True' 7 times slower
8     if (str == "true") {
9         return true;
10    }
11    if (str == null) {
12        return false;
13    }
14    switch (str.length()) {
15        case 2: {
16            char ch0 = str.charAt(0);
17            char ch1 = str.charAt(1);
18            return
19                (ch0 == 'o' || ch0 == 'O') &&
20                (ch1 == 'n' || ch1 == 'N');
21        }
22        case 3: {
23            char ch = str.charAt(0);
24            if (ch == 'y') {
25                return
26                    (str.charAt(1) == 'e' || str.charAt(1) == 'E') &&
27                    (str.charAt(2) == 's' || str.charAt(2) == 'S');
28            }
29            if (ch == 'Y') {
30                return
31                    (str.charAt(1) == 'e' || str.charAt(1) == 'E') &&
32                    (str.charAt(2) == 's' || str.charAt(2) == 'S');
33            }
34            // Correto: return false;
35        }
36        case 4: {
37            char ch = str.charAt(0);
38            if (ch == 't') {
39                return
40                    (str.charAt(1) == 'r' || str.charAt(1) == 'R') &&
41                    (str.charAt(2) == 'u' || str.charAt(2) == 'U') &&
42                    (str.charAt(3) == 'e' || str.charAt(3) == 'E');
43            }
44            if (ch == 'T') {
45                return
46                    (str.charAt(1) == 'r' || str.charAt(1) == 'R') &&
47                    (str.charAt(2) == 'u' || str.charAt(2) == 'U') &&
48                    (str.charAt(3) == 'e' || str.charAt(3) == 'E');
49            }
50        }
51    }
52    return false;
53 }
```

Linha	Nó	Código
1	1	float avg(float[] array, int length)
2	1	{
3	1	int i = 0;
4	1	float avg = 0;
5	1	float sum = 0;
6	2	while(i < length)
7	3	{
8	3	sum += array[i];
9	3	i++;
10	3	}
11	4	if(length > 0)
12	5	{
13	5	avg = (sum / length);
14	5	}
15	6	return avg;
16	6	}

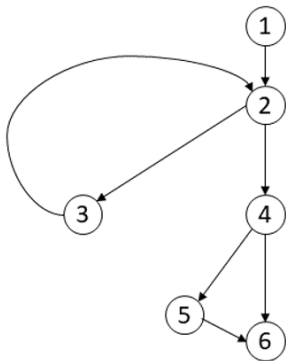


Figura 1: Programa avg

Programa com defeito			Conjunto de testes				Espectro							
Linha	Nó	Código	T _n	Teste	Saída esperada	Saída obtida	Linha	Nó	t ₁	t ₂	t ₃	t ₄	t ₅	t ₆
1	1	public static int imc(double peso, double altura)	t ₁	imc(55.0, 1.75)	0	0	1	1	•	•	•	•	•	•
2	1	{	t ₂	imc(75.0, 1.75)	1	1	2	1	•	•	•	•	•	•
3	1	double imc;	t ₃	imc(90.0, 1.75)	2	2	3	1	•	•	•	•	•	•
4	1	int nivel;	t ₄	imc(100.0, 1.75)	3	3	4	1	•	•	•	•	•	•
5	1	imc = peso/(altura*altura);	t ₅	imc(110.0, 1.75)	4	5	5	1	•	•	•	•	•	•
6	2	if (imc < 18.5)	t ₆	imc(150.0, 1.75)	5	4	6	2	•	•	•	•	•	•
7	3	{					7	3	•					
8	3	nivel = 0;					8	3	•					
9	3	}					9	3	•					
10	4	else if (imc < 25)					10	4		•	•	•	•	•
11	5	{					11	5		•				
12	5	nivel = 1;					12	5		•				
13	5	}					13	5		•				
14	6	else if (imc < 30)					14	6			•	•	•	•
15	7	{					15	7			•			
16	7	nivel = 2;					16	7			•			
17	7	}					17	7			•			
18	8	else if (imc < 35)					18	8				•	•	•
19	9	{					19	9				•		
20	9	nivel = 3;					20	9				•		
21	9	}					21	9				•		
22	10	else if (imc > 40) \\(imc < 40)					22	10				•	•	
23	11	{					23	11					•	
24	11	nivel = 4;					24	11					•	
25	11	}					25	11					•	
26	12	else					26	12						•
27	12	{					27	12						•
28	12	nivel = 5;					28	12						•
29	12	}					29	12						•
30	13	return nivel;					30	13	•	•	•	•	•	•
31	13	}					31	13	•	•	•	•	•	•

Figura 2: Espectro do programa imc

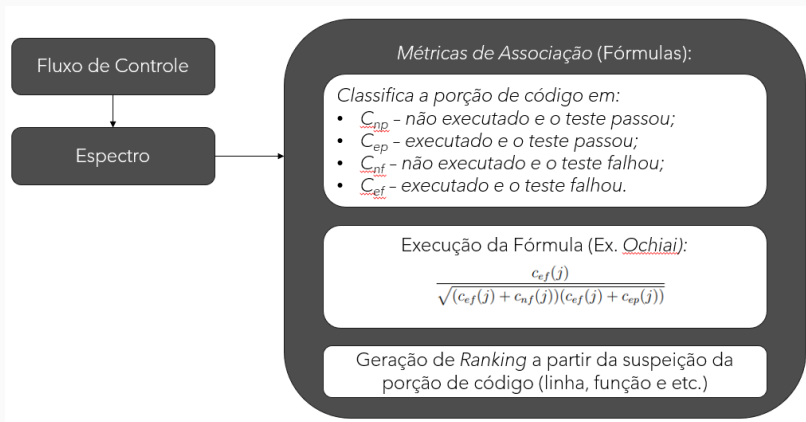


Figura 3: Métricas de associação

LEVANTAMENTO DE RANKING DO PROGRAMA IMC

Linha	Nó	Código	Linha	Nó	t ₁	t ₂	t ₃	t ₄	t ₅	t ₆	c _{np}	c _{ep}	c _{nf}	c _{ef}	Ochiai
1	1	public static int imc(double peso, double altura)	1	1	•	•	•	•	•	•	0	4	0	2	0
2	1	{	2	1	•	•	•	•	•	•	0	4	0	2	0.57
3	1	double imc;	3	1	•	•	•	•	•	•	0	4	0	2	0.57
4	1	int nivel;	4	1	•	•	•	•	•	•	0	4	0	2	0.57
5	1	imc = peso/(altura*altura);	5	1	•	•	•	•	•	•	0	4	0	2	0.57
6	2	if (imc < 18.5)	6	2	•	•	•	•	•	•	0	4	0	2	0.57
7	3	{	7	3	•						3	1	2	0	0
8	3	nivel = 0;	8	3	•						3	1	2	0	0
9	3	}	9	3	•						3	1	2	0	0
10	4	else if (imc < 25)	10	4		•	•	•	•	•	1	3	0	2	0.63
11	5	{	11	5		•					3	1	2	0	0
12	5	nivel = 1;	12	5		•					3	1	2	0	0
13	5	}	13	5		•					3	1	2	0	0
14	6	else if (imc < 30)	14	6			•	•	•	•	2	2	0	2	0.70
15	7	{	15	7			•				3	1	2	0	0
16	7	nivel = 2;	16	7			•				3	1	2	0	0
17	7	}	17	7			•				3	1	2	0	0
18	8	else if (imc < 35)	18	8				•	•	•	3	1	0	2	0.81
19	9	{	19	9				•			3	1	2	0	0
20	9	nivel = 3;	20	9				•			3	1	2	0	0
21	9	}	21	9				•			3	1	2	0	0
22	10	else if (imc > 40) \\(imc < 40)	22	10					•	•	4	0	0	2	1
23	11	{	23	11					•	4	0	1	1	1	0.70
24	11	nivel = 4;	24	11					•	4	0	1	1	1	0.70
25	11	}	25	11					•	4	0	1	1	1	0.70
26	12	else	26	12					•	4	0	1	1	1	0.70
27	12	{	27	12					•	4	0	1	1	1	0.70
28	12	nivel = 5;	28	12					•	4	0	1	1	1	0.70
29	12	}	29	12					•	4	0	1	1	1	0.70
30	13	return nivel;	30	13	•	•	•	•	•	•	0	4	0	2	0.57
31	13	}	31	13	•	•	•	•	•	•	0	4	0	2	0.57

Figura 4: Levantamento de ranking do programa IMC

Jaguar 2 (J*Av*a c*overa*g*e* f*ault* l*ocalization* R*anking* 2) é uma ferramenta de código aberto que utiliza informações de fluxo de controle e fluxo de dados para localização de defeitos baseado em espectro em aplicações Java.

Jaguar Portal é uma plataforma projetada para recepcionar, armazenar e disponibilizar informações resultantes da utilização de *SBFL* em um ambiente de integração contínua.

A plataforma é composta por um conjunto de aplicações descritas a seguir:

1. *jaguarportal-action*, uma ação do *Github Actions*;
2. *jaguarportal-submit*, uma linha de comando responsável por capturar o arquivo com os dados da técnica *SBFL* gerado pela *Jaguar 2* e submetê-lo ao *jaguarportal-web*, ela é executada pela *jaguarportal-action* durante a execução da esteira de *CI*;
3. *jaguarportal-web* é uma aplicação web que possui um conjunto de pontos de extremidades responsáveis por recepcionar e armazenar os dados em banco de dados e em páginas da web para exibição das análises de *SBFL*.

JAGUAR PORTAL - DIAGRAMA

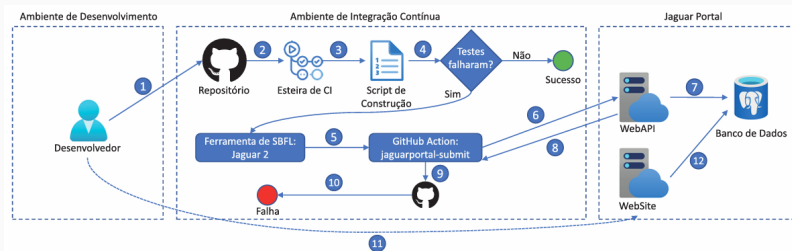


Figura 5: Diagrama de implantação da Jaguar Portal em um ambiente de CI.

JAGUAR PORTAL - BUILD FAIL

Quando uma falha ocorre em uma build, é gravado no *annotations* o *top 10* de suspeição e o link para acesso aos dados na Jaguar Portal.

Re-run triggered 3 weeks ago
ericksonlBs #3 [test2](#)

Status: **Failure**

Total duration: **1m 38s**

Artifacts: **1**

maven.yml
on: pull_request

build 1m 27s

Annotations
1 error and 10 notices

build
Process completed with exit code 1.

① **Jaguar Portal Analysis**
<https://jaguarportal.azurewebsites.net/Analyzes/Detail/17>

① 1.0000 - br/usp/each/ericksonlBs/imc/IMC.java - Line: 30 (SBFL RANKING)
/src/main/java/IMC.java:CEP:0 CFP:0 CIP:0

① 0.8165 - br/usp/each/ericksonlBs/imc/IMC.java - Line: 26 (SBFL RANKING)
/src/main/java/IMC.java:CEP:12 CFP:1 CIP:0

① 0.7071 - br/usp/each/ericksonlBs/imc/IMC.java - Line: 22 (SBFL RANKING)
/src/main/java/IMC.java:CEP:12 CFP:2 CIP:0

Figura 6: Build que falhou no GitHub.

JAGUAR PORTAL - PULL REQUEST

Se o *commit* foi efetuado em um *pull request*, as informações também são inseridas na conversa do *pull request* do GitHub.

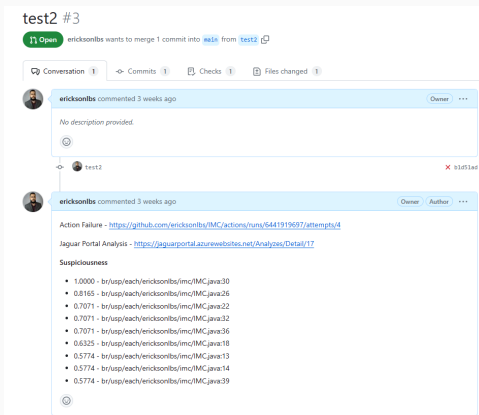


Figura 7: Pull request no GitHub.

Ao acessar os links demonstrados na figuras 6 e 7, é aberto uma página web da *Jaguar Portal* com a análise *SBFL* do programa em questão. Os dados são apresentados em um ranking dos trechos de código mais suspeitos do programa. Conforme apresentado na figura 8.

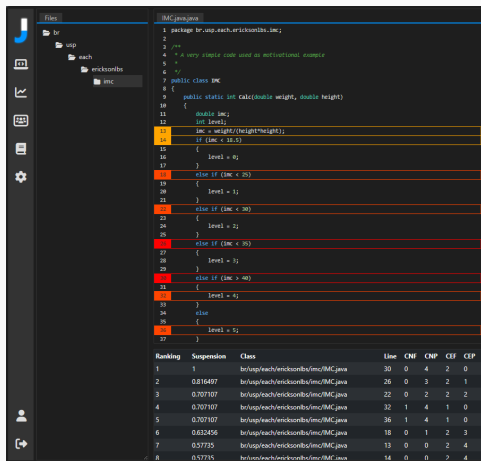


Figura 8: Visualização da análise SBFL do programa IMC na Jaguar Portal Web

A participação desta atividade é voluntária, porém, para que seja possível a participação, o participante ***obrigatoriamente*** deverá:

1. Preencher o questionário on-line de Avaliação de Perfil
(ACESSAR QUESTIONÁRIO)
2. Preencher e enviar o "Termo De Consentimento Livre e Esclarecido" via e-mail (ericksonlbs@usp.br e chaim@usp.br)
(BAIXAR TERMO)

A atividade consiste em simular um ambiente de desenvolvimento de software que adota a prática de desenvolvimento orientado por testes (*Test Driven Development - TDD*).

Será disponibilizado um endereço para acesso remoto em uma máquina virtual que conterá o Visual Studio Code para o desenvolvimento da tarefa.

Será efetuado uma **gravação da tela** durante o acesso da máquina virtual, para que seja possível posteriormente a análise por meio de vídeo.

Será compartilhado individualmente um repositório com uma versão específica do projeto Jsoup para cada participante. Neste repositório terá um *issue* cadastrado, informando sobre um possível defeito do software.

Você terá que identificar e corrigir o defeito, tendo suporte da ferramenta *Jaguar Portal* com informações de *SBFL*, seguindo a sequência de tarefas demonstradas no próximo slide.

A atividade consiste nas seguintes tarefas:

1. Criar um *branch* do repositório *main*;
2. Criar um teste que falhe com a situação de 'defeito' informada no *issue* do *GitHub*, efetuar um *commit* e submeter um *pull-request*.
3. Após a falha do teste, será acrescentado um comentário no *pull-request* com o link da *Jaguar Portal*, onde será possível, por meio das credenciais que lhe foi enviada anteriormente, visualizar informações de suspeição do defeito.
4. Identificar e corrigir o defeito, e submeter um novo *commit*.
5. O esperado é que o teste falhado anteriormente, agora não falhe mais, logo, o defeito reportado na *issue* tenha sido resolvido.

- A avaliação desta atividade não considerará se o participante chegou na resolução do problema ou se a resolução está correta ou incorreta, pois o intuito é observar a forma em que são utilizadas as ferramentas;

- A atividade deverá ser efetuada em uma única sessão de até 4h;
- Toda a tarefa deverá ser executada dentro da máquina remota disponibilizada;
- O acesso a máquina será por meio do Protocolo de Desktop Remoto (RDP), com dados enviados a cada participante previamente;
- O horário da sessão será agendada, de acordo com a disponibilidade dos participantes.